

1 SDK简介 【Introduction to SDK】

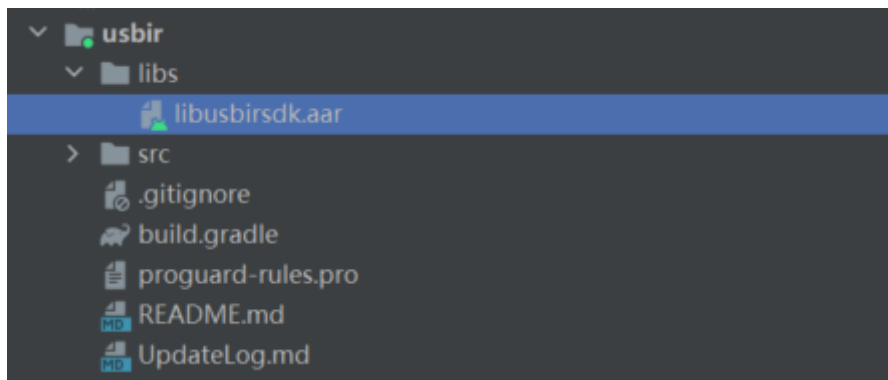
Falcon Android SDK 实现了Android系统下Falcon系列红外摄像头应用接口，为客户提供参考demo和底层封装库方面客户二次开发，后面章节统一称为 SDK。

【The Falcon Android SDK implements the Falcon series infrared camera application interface under the Android system, and provides customers with reference demos and the secondary development of the underlying package library. The following chapters are collectively referred to as SDK.】

2 SDK引入 【SDK introduction】

2.1 步骤一 【Step 1】

把aar库放在工程目录下 【Put the aar library in the project directory】



2.2 步骤二 【Step 2】

在build.gradle下添加库目录 【Add the library directory under build.gradle】

```
1 dependencies {  
2     // use jar or aar  
3     implementation fileTree(dir: 'libs', include: ['*.jar','*.aar'])  
4     ...  
5 }
```

3 APIs

SDK JAVA层提供6个Java类库 LibIRTemp、LibIRParse、LibIRProcess、IRCMD、UVCCamera、DualUVCCamera。

【SDK JAVA layer provides 6 Java class libraries LibIRTemp, LibIRParse, LibIRProcess, IRCMD, UVCCamera, DualUVCCamera.】

每个库的作用如下： 【The role of each library is as follows:】

备注：这里列出的API接口仅做了简单说明，方便用户对SDK的结构有大概了解，且不同的SDK使用到的具体接口会有差异。接口的请求参数或返回值，以及详细注释，请参考doc中的文档。

【Remarks: The API interfaces listed here are only briefly explained, so that users can have a general understanding of the structure of the SDK, and the specific interfaces used by different SDKs will be different. For the request parameters or return values of the interface, as well as detailed comments, please refer to the documentation in the doc.】

3.1 UVCCamera相关

用于UVC相机设备的寻找、设备插拔监听，连接打开和出图回调，以及发送对应的控制命令

【It is used to search for UVC camera equipment, monitor the plug-in and unplug of the equipment, open the connection and call back the picture, and send the corresponding control commands】

3.1.1 初始化

```
1  ..
2
3  // UVCCamera init
4  ConcreateUVCBuilder concreateUVCBuilder = new ConcreateUVCBuilder();
5  uvccamera = concreateUVCBuilder
6              .setUVCType(UVCType.xxx)
7              .setOutputwidth(cameraWidth)
8              .setOutputHeight(cameraHeight)
```

设置不同的cameraWidth和cameraHeight可以控制不同的出图分辨率

- cameraWidth:256,cameraHeight:384,红外+温度
- cameraWidth:256,cameraHeight:192,默认为红外

调用startY16ModePreview接口，传入不同的参数可以获取不同的Y16格式数据流

3.1.2 接口

具体接口及返回值请参考 doc 中文档。【For specific interfaces and return values, please refer to the documentation in doc.】

3.2 IRCMD相关

与机芯固件进行通讯接口，实现信息读取和写入，实现机芯配置功能和固件升级等。

【It communicates with the movement firmware to realize information reading and writing, realize movement configuration functions and firmware upgrades, etc.】

SDK采用安卓软件库AAR格式发布。

【The SDK is released in the AAR format of the Android software library.】

3.2.1 初始化

```
1  ..
2
3  // IRCMD init
4  ConcreteIRCMDBuilder concreteIRCMDBuilder = new ConcreteIRCMDBuilder();
5  ircmd = concreteIRCMDBuilder
6      .setIrcmdType(IRCMDType.xxx)
7      .setIdCamera(uvcCamera.getNativePtr())
8      .build();
```

3.2.2 接口

具体接口及返回值请参考 doc 中文档。【For specific interfaces and return values, please refer to the documentation in [doc](#).】

3.3 LibIRParse相关

对原始数据进行切分（为温度和图像），以及数据的格式转换，如RGB和YUV之间的转换

【Segment the original data (temperature and image), and data format conversion, such as conversion between RGB and YUV】

3.3.1 接口

具体接口及返回值请参考 doc 中文档。【For specific interfaces and return values, please refer to the documentation in [doc](#).】

3.4 LibIRProcess相关

一些额外的处理算法，如伪彩色、旋转镜像翻转等

【Some additional processing algorithms, such as false color, rotating mirror flip, etc.】

3.4.1 接口

具体接口及返回值请参考 doc 中文档。【For specific interfaces and return values, please refer to the documentation in [doc](#).】

3.5 LibIRTemp相关

对温度数据的解析和处理，以及温度数据的二次修正等计算

【Analysis and processing of temperature data, and calculation of secondary correction of temperature data】

3.5.1 接口

具体接口及返回值请参考 doc 中文档。【For specific interfaces and return values, please refer to the documentation in [doc](#).】

3.6 DualUVCCamera相关

3.6.1 初始化

```
1  ..
2
3  // DualUVCCamera 初始化
4  ConcreateDualBuilder concreateDualBuilder = new ConcreateDualBuilder();
5  dualUVCCamera = concreateDualBuilder
6      .setDualType(DualType.xxx)
7      .setIRSize(xxx, xxx)
8      .setVLSize(xxx, xxx)
9      .setDualSize(xxx, xxx)
10     .setDataFlowMode(dataFlowMode)
11     .build();
```

3.6.2 接口

具体接口及返回值请参考 doc 中文档。【For specific interfaces and return values, please refer to the documentation in `doc`.】

3.7 USBMonitor

用来监听设备的插拔。

3.7.1 接口

具体接口及返回值请参考 doc 中文档。【For specific interfaces and return values, please refer to the documentation in `doc`.】

4 APP开发参考指南【APP Development Reference Guide】

4.1 APP出图和测温流程简介【Introduction to APP drawing and temperature measurement process】

- USBcamera处理类IRUVC实现摄像头插入状态监听、USB打开权限控制、摄像头参数初始化、摄像头连接预览命令和数据采集回调。在onConnect回调里面打开摄像头，在回调函数onFrame里面实现图像数据和温度数据获取拷贝分割和旋转等操作。

【The USBcamera processing class IRUVC realizes the monitoring of camera insertion status, USB opening authority control, camera parameter initialization, camera connection preview command, and data collection callback. Turn on the camera in the onConnect callback, and implement operations such as image data and temperature data acquisition, copying, segmentation, and rotation in the callback function onFrame.】

- 图像数据经过回调采集送到ImageThread进行图像处理和转换操作得到ARGB格式。最后CameraView 使用bitmap把ARGB格式数据放大填充屏幕进行图像绘制。

【The image data is collected by callback and sent to ImageThread for image processing and conversion operations to obtain the ARGB format. Finally, CameraView uses bitmap to enlarge the ARGB format data to fill the screen for image drawing.】

- TemperatureView实现点、线、框温度测量显示，onTouch方法里面实现了手指绘制点、线、框并记录坐标，测温进程调用测温库实现了温度测量，最终显示在相应坐标。

【TemperatureView realizes the temperature measurement and display of points, lines, and boxes. The onTouch method realizes the finger drawing points, lines, and boxes and records the coordinates. The temperature measurement process calls the temperature measurement library to realize the temperature measurement, and finally displays the corresponding coordinates.】

- 机芯配置命令，红外机芯固件提供一下接口实现图像和测试参数的配置，可以提高图像和测温质量。类Libircmd提供一系列native函数调用，需要在打开摄像头的情情况下传入摄像头句柄指针调用。

【Movement configuration command, infrared movement firmware provides the following interface to realize the configuration of image and test parameters, which can improve the quality of image and temperature measurement. The class Libircmd provides a series of native function calls, which need to be called with the camera handle pointer when the camera is turned on.】

4.2 设备pid过滤【Device pid filter】

UVC芯片pid会有默认值，如果您的设备pid不是该默认值，需要更改实际的pid过滤参数。

【UVC chip pid will have a default value. If your device pid is not the default value, you need to change the actual pid filter parameters.】

4.3 测温功能【Temperature measurement function】

4.3.1 Libirtemp库测温【Lipirtemp library temperature measurement】

- RAW温度数据摄氏度计算：【RAW temperature data Celsius calculation:】

x y为坐标数据，cameraWidth为摄像头像素宽度，如果数据经过旋转，需要使用旋转后的宽度。

【x y is the coordinate data, cameraWidth is the pixel width of the camera, if the data is rotated, the rotated width needs to be used.】

```
1 (temperature[(x*camerawidth+y)*2]+((int)(temperature[(x*camerawidth+y)*2+1])<<8))/64 -273.15;
```

- 测温库提供点线框测温接口【Temperature measurement library provides point wire frame temperature measurement interface】

调用Libirtemp的setTempData方法，传入温度数据，然后就可以调用对应的方法获取温度信息

```
1 /**  
2  * copy Temperature data from buffer<br/>  
3  *  
4  * @param src Temperature buffer  
5  */  
6 public void setTempData(byte[] src)
```

- 返回坐标点温度值摄氏度

【Returns the temperature value of the coordinate point in Celsius】

```
1 Libirtemp.getTemperatureOfPoint(Point point)
```

- 返回线的最低温、最高温、平均温、最低温坐标、最高温坐标

【Return line's lowest temperature, highest temperature, average temperature, lowest temperature coordinate, highest temperature coordinate】

```
1 Libirtemp.getTemperatureOfLine(Line line)
```

- 返回矩形的最低温、最高温、平均温、最低温坐标、最高温坐标

【Return the lowest temperature, highest temperature, average temperature, lowest temperature coordinates, and highest temperature coordinates of the rectangle】

```
1 Libirtemp.getTemperatureOfRect(Rect rect)
```

4.3.2 IRCMD库测温【IRCMD library temperature measurement】

- 返回坐标点温度值摄氏度

【Returns the temperature value of the coordinate point in Celsius】

```
1 public int getPointTemperatureInfo(int pixelPointX, int pixelPointY, int[]  
   temperatureValue)
```

- 返回线的最低温、最高温、平均温、最低温坐标、最高温坐标

【Return line's lowest temperature, highest temperature, average temperature, lowest temperature coordinate, highest temperature coordinate】

```
1 public int getLineTemperatureInfo(int startPointX, int startPointY, int  
   endPointX, int endPointY, int[] temperatureValue)
```

- 返回矩形的最低温、最高温、平均温、最低温坐标、最高温坐标

【Return the lowest temperature, highest temperature, average temperature, lowest temperature coordinates, and highest temperature coordinates of the rectangle】

```
1 public int getRectTemperatureInfo(int startPointX, int startPointY, int  
   endPointX, int endPointY,  
2                                     int[] temperatureValue)
```

- 获取整帧最小温度【Get the min temperature of the frame】

```
1 public int getCurrentFrameMinTemperature(int[] temperatureValue)
```

- 获取整帧最大温度【Get the maximum temperature of the frame】

```
1 public int getCurrentFrameMaxTemperature(int[] temperatureValue)
```

- 获取整帧的最大最小温度信息【Get the maximum and minimum temperature information of the frame】

```
1 public int getCurrentFrameMaxAndMinTemperature(int[] temperatureValue)
```

4.4 测温二次修正【Second correction of temperature measurement】

具体内容见文档 `用户开发标定 User calibration instructions\环境变量修正Ambient variable correction\环境变量修正Ambient variable correction.pdf`

【For details, see the document `User calibration instructions\Ambient variable correction\Ambient variable correction.pdf`】

4.5 高低增益自动切换【High and low gain automatic switching】

```
1 ...
2 private LibIRProcess.AutoGainSwitchInfo_t auto_gain_switch_info = new
  LibIRProcess.AutoGainSwitchInfo_t();
3 private LibIRProcess.GainSwitchParam_t gain_switch_param = new
  LibIRProcess.GainSwitchParam_t();
4 ...
5
6 // 自动增益切换参数auto gain switch parameter
7 gain_switch_param.above_pixel_prop = 0.1f; //用于high -> low gain,
  设备像素总面积的百分比
8 gain_switch_param.above_temp_data = (int) ((130 + 273.15) * 16 * 4);
  //用于high -> low gain,高增益向低增益切换的触发温度
9 gain_switch_param.below_pixel_prop = 0.95f; //用于low -> high gain,
  设备像素总面积的百分比
10 gain_switch_param.below_temp_data = (int) ((110 + 273.15) * 16 *
  4);//用于low -> high gain,低增益向高增益切换的触发温度
11 auto_gain_switch_info.switch_frame_cnt = 5 * 15; //连续满足触发条件帧数
  超过该阈值会触发自动增益切换(假设出图速度为15帧每秒,则5 * 15大概为5秒)
12 auto_gain_switch_info.waiting_frame_cnt = 7 * 15; //触发自动增益切换之
  后,会间隔该阈值的帧数不进行增益切换监测(假设出图速度为15帧每秒,则7 * 15大概为7秒)
13
14 ...
15
16 /**
17  * 高低增益自动切换【High and low gain automatic switching】<br/>
18  * 每一帧都进行处理<br/>
19  *
20  * @param temp_frame 原始的温度数据<br/>
21  * @param image_res 原始温度数据的宽高<br/>
22  * @param auto_gain_switch_info 触发自动增益切换帧数阈值,当监测到在
  gain_switch_param条件下高增益或低增益的帧数超过auto_gain_switch_info时则触发自动增益
  切换<br/>
23  * @param gain_switch_param 自动增益切换参数<br/>
24  * @param autoGainSwitchCallback 自动切换回调
25  * @return see {@link IrcmdResult}
26  */
```

```

27     public int autoGainSwitch(byte[] temp_frame, LibIRProcess.ImageRes_t
image_res,
28                                     LibIRProcess.AutoGainSwitchInfo_t
auto_gain_switch_info,
29                                     LibIRProcess.GainSwitchParam_t
gain_switch_param,
30                                     AutoGainSwitchCallback autoGainSwitchCallback)

```

调用方式在IRUVC onframe回调里面每一帧调用即可。【The calling method can be called every frame in the IRUVC onframe callback.】

4.6 过曝保护【Overexposure protection】

```

1  ...
2  // 防灼烧参数over_portect parameter
3      int low_gain_over_temp_data = (int) ((550 + 273.15) * 16 * 4); //低增
    益下触发防灼烧的温度
4      int high_gain_over_temp_data = (int) ((150 + 273.15) * 16 * 4); //高
    增益下触发防灼烧的温度
5      float pixel_above_prop = 0.02f; //设备像素总面积的百分比
6      int switch_frame_cnt = 7 * 15; //连续满足触发条件超过该阈值会触发防灼烧(假设
    出图速度为15帧每秒, 则7 * 15大概为7秒)
7      int close_frame_cnt = 10 * 15; //触发防灼烧之后, 经过该阈值的帧数之后会解除
    防灼烧(假设出图速度为15帧每秒, 则10 * 15大概为10秒)
8  ...
9
10 /**
11     * 过曝保护【Overexposure protection】<br/>
12     * 每一帧都进行处理<br/>
13     *
14     * @param gainStatus          当前的增益状态{@link
    CommonParams.GainStatus#HIGH_GAIN}{@link CommonParams.GainStatus#LOW_GAIN}
    <br/>
15     * @param temp_frame          原始的温度数据<br/>
16     * @param image_res           原始温度数据的宽高<br/>
17     * @param low_gain_over_temp_data 低增益下触发防灼烧温度值<br/>
18     * @param high_gain_over_temp_data 高增益下触发防灼烧温度值<br/>
19     * @param pixel_above_prop     pixels above the maxtemp, 一帧中像素所
    占的比例大于该阈值会触发防灼烧<br/>
20     * @param switch_frame_cnt     触发防灼烧的连续帧数<br/>
21     * @param close_frame_cnt      触发防灼烧之后帧数达到该阈值之后会打开快门
    <br/>
22     * @param avoidOverexposureCallback 防灼烧回调, true:触发防灼烧; false:防灼烧
    解除
23     * @return see {@link IrcmdResult}
24     */
25     public int avoidOverexposure(CommonParams.GainStatus gainStatus, byte[]
temp_frame, LibIRProcess.ImageRes_t image_res,
26                                     int low_gain_over_temp_data, int
high_gain_over_temp_data, float pixel_above_prop,
27                                     int switch_frame_cnt, int close_frame_cnt,
    AvoidOverexposureCallback avoidOverexposureCallback)

```


调用方式在IRUVC onframe回调里面每一帧调用即可。【The calling method can be called every frame in the IRUVC onframe callback.】

4.7 不同模式数据流【Different data flows】

切换不同的出图模式会输出不同分辨率和模式的数据流。

数据流【dataFlow】	数据【data】	分辨率【resolution】
IMAGE_AND_TEMP_OUTPUT	图像+温度【Image + Temp】	256*384
IMAGE_OUTPUT	图像【Image】	256*192
TEMP_OUTPUT	温度(通过startY16ModePreview切换) 【Temp(Toggle via startY16ModePreview)】	256*192

在 `uvccamera` 初始化的时候，传入不同的分辨率会切换到不同的数据流模式：

```
1 uvccamera = new UVCCamera(cameraWidth, cameraHeight);
2 /**
3      * cameraWidth:256,cameraHeight:384,图像+温度
4      * cameraWidth:256,cameraHeight:192,图像
5      * cameraWidth:256,cameraHeight:192,(调用startY16ModePreview, 传入
6      Y16_MODE_TEMPERATURE)温度
7      */
```